BOTS.BUSINESS · BJS COMMAND PACK

Advance Poll Bot V2

The complete, production-ready rebuild of your Telegram poll experience — a single self-updating Smart Bubble, a multi-channel sponsor gateway, automatic join-left protection, a cart-style step-by-step builder and a full history dashboard. This guide walks you from zero to your first live poll in about ten minutes.

Setup Guide · Operations Manual · Troubleshooting

- ✦ **Zero-Spam Smart Bubble** — one message, every menu
- ✦ **Sponsor Gateway** — up to 2 channels before a vote counts
- ✦ **Join-Left Protection** — leavers' votes removed automatically
- ✦ **History Dashboard** — pause · resume · stop · hide votes
- ✦ **Auto-Cleaning Database** — self-maintaining storage

OCTOBER 2026 · VERSION 2.0 · BUILT FOR BOTS.BUSINESS

CHAPTER 01

Welcome to V2 🚀

Advance Poll Bot V2 is a complete rebuild of the classic poll bot, designed to feel like a real app inside Telegram. Instead of flooding the chat with messages, the bot runs on a **single dynamic "Smart Bubble"** that updates itself in place as you navigate menus, the builder and the dashboard. Polls are posted to your channel as one clean message with inline vote buttons, live result bars and a refresh control.

This package contains the entire bot as a **Bots.Business BJS command pack** — 42 ready-to-import commands covering navigation, the poll builder, the voting engine, the sponsor gateway, re-verification sweeps, the dashboard and the auto-clean watchdog. You do not need to write any code: import the pack, connect your bot, add it as an admin in your channels and you are live. Every behaviour described in this guide is already implemented and was verified end-to-end with an automated 37-check test suite before delivery.

🧹 Zero-Spam Smart UI

All bot menus live inside one self-updating message in your private chat. Navigating back and forth never creates new messages — the same bubble simply re-renders, exactly like an app screen.

💬 Multi-Channel Sponsor Gateway

Attach up to 2 sponsor channels to any poll. Users must join them before their vote is counted — cross-promotion that actually converts, built into the voting flow itself.

👮 Join-Left Protection

Every vote is re-verified when the voter interacts with the poll and again in scheduled background sweeps. If a voter leaves a sponsor channel, the bot silently removes the vote and updates the message.

🎧 Step-by-Step Poll Builder

Type your title, then your options one by one into a cart-style list — no cryptic symbols or formats. Skip buttons, remove-last and automatic limits keep the flow friendly on any device.

✈ Advanced History Dashboard

Pause, resume, stop, hide vote counts, re-verify all votes, re-post or delete — every action is one tap in the inline panel, and changes apply instantly across every channel.

🧹 Auto-Cleaning Database

An hourly watchdog stops over-age polls, re-verifies live ones and deletes stopped polls 48 hours after they end, keeping your storage tidy and your dashboard fast without any manual work.

CHAPTER 02

What's in the Package

The deliverable is a **Git-importable Bots.Business repository**. The folder follows the exact repository format documented at `help.bots.business`, so the web app can clone it directly. Inside, `bot.json` is the manifest the importer requires, and the `commands/` folder holds one file per command — each file is plain, human-readable BJS with a `/*CMD … CMD*/` header, which also makes it perfect for manual copy-paste if you prefer that route.

```
advance-poll-bot-v2/
├── bot.json                  ← repo manifest (bb_sync_version 1.0)
└── commands/                 ← 42 BJS command files
    ├── _start.js             ← Smart Bubble main menu
    ├── _np*.js               ← step-by-step poll builder (10 cmds)
    ├── _vote*.js  _refresh.js← voting engine + gateway (6 cmds)
    ├── _pollAdmin.js  _myPolls.js  _pollAction.js
    │                         ← history dashboard (3 cmds)
    ├── _sweepPoll.js  _sweepNext.js  _cleanSweep.js
    │   _doCleanup.js         ← protection + auto-clean (4 cmds)
    └── _render*.js  …        ← Smart Bubble renderer & helpers
```

You receive this folder unpacked **and** zipped (`advance-poll-bot-v2.zip`), plus this guide. The zip is what you upload to GitHub or extract locally; the unpacked folder is convenient for browsing and for the manual paste route. A companion file `advance-poll-bot-v2-guide.html` is the editable source of this very document.

Command group	Key commands	What it powers
Navigation	`/start`, `/help`, `/about`	Smart Bubble menu, help, about pages
Poll builder	`/newpoll`, `/npTitle` … `/npPublish`	Cart-style step-by-step creation flow
Voting engine	`/vote`, `/refresh`, `/pollRender`	Gateway checks, counting, live result updates
Dashboard	`/mypolls`, `/pollAdmin`, `/pollAction`	History list and per-poll controls
Protection	`/sweepPoll`, `/sweepNext`, `/voteCheck`	Membership re-verification sweeps
Maintenance	`/cleanSweep`, `/doCleanup`, `/setcmds`	Watchdog, data cleanup, command list

CHAPTER 03

Before You Begin — Requirements

You need three things, and none of them cost anything. Take two minutes to prepare them now and the rest of the setup becomes a smooth, uninterrupted run. The free Bots.Business plan (10 million iterations per month) is more than enough for this bot — a full vote with a two-channel gateway costs only a handful of iterations.

1 · A Bots.Business account 🏢

Sign in at `bots.business` with the web app (or install the Android app). Create a new bot in the app — you can leave it empty; the import will fill it with all 42 commands. The web app is strongly recommended for the Git import route because its import dialog is the easiest to use.

2 · A Telegram bot token 🤖

In Telegram, open `@BotFather`, send `/newbot` and follow the two prompts (display name and username). BotFather replies with a token like `123456:AA…`. Paste that token into your Bots.Business bot's settings so the platform can drive it. Keep the token private — anyone holding it controls your bot.

3 · Channels with the right admin rights 📣

The bot must be an **admin** of your **posting channel** with the *Post Messages* permission — that is how it publishes poll messages. For every **sponsor channel** you attach, add the bot as admin too (no posting rights needed) because membership checks via `getChatMember` only work for bots that are inside the channel. Sponsor channels must be public (`@username`) so the gateway can generate join buttons.

⚠️ **Why admin rights matter:** the gateway is only as reliable as its source of truth. If the bot is not an admin in a sponsor channel, Telegram refuses membership lookups and votes would either be blocked or unverifiable — so the bot asks for admin *before* letting the flow start.

CHAPTER 04

Installation — Two Routes

Pick **Route A (Git import)** if you can spare five minutes on GitHub — it installs all 42 commands in one shot. Pick **Route B (manual paste)** if you prefer to stay inside the Bots.Business app. Both routes produce the identical bot; you can even start with B and switch to A later. Either way, run the import on the bot you created in Chapter 3.

ROUTE A

Git Import (recommended)

1. **Create an empty GitHub repository**
   On `github.com` click **New repository**, name it `advance-poll-bot-v2`, keep it **Public** and do not add a README. Public read access is all the importer needs — never put your bot token in the repo.

2. **Upload the two items**
   Use GitHub's **uploading an existing file** or **Add file → Upload files** to add `bot.json` and the whole `commands/` folder from the unzipped package, preserving the folder structure, then commit. The final layout must match the tree in Chapter 2 exactly.

3. **Open Git import in Bots.Business**
   In the web app, open your bot's command list and locate the **Git** / **Import from Git** option (also described at `help.bots.business → Import and export a bot with Git`). Paste your repository URL, keep the default branch (`main` or `master`) and start the import.

4 Confirm the result

The importer validates `bb_sync_version`, replaces the bot's commands with the repository's 42 files and reports success. Open the command list and verify you see `/start`, `/npMenu`, `/cleanSweep` and friends.

> ⚠️ **Important:** a Git import **replaces all existing commands and installed libraries** of the destination bot. Run it on a fresh, empty bot — or export a backup of the current bot first, exactly as the official documentation advises.

ROUTE B

Manual Paste

1 Create the commands

In the app's BJS editor, add a new command for each file. Set the command name from the `command:` header (e.g. `/start`) and turn on **Wait for answer / need_reply** for the six builder commands listed in its header (`/npTitle`, `/npDesc`, `/npOpt`, `/npChannel`, `/npSponsor`, `/npSponsor2`).

2 Paste the code bodies

Copy everything below the `CMD*/` line of each file into the command's BJS body and save. Aliases go into the alias field: `/start` also answers `/menu`, `/npMenu` also answers `/newpoll`, `/myPolls` also answers `/mypolls`, `/helpPage` also answers `/help`.

3 Double-check the six flags

The need_reply flags are what make the typed-input builder work. If a later step ever "doesn't listen", revisit that command's settings — 99% of the time the flag is simply off.

CHAPTER 05

First Run ✅

With commands installed and the token connected, testing takes under a minute. Open your bot in Telegram and send `/start`. You should immediately see the dark-themed welcome bubble with four buttons: 🆕 **Create Poll**, 📊 **My Polls**, ❓ **Help** and ℹ️ **About**. Tap through them — notice that every tap re-renders the *same* message instead of sending new ones. That is the Smart Bubble working.

Then send `/setcmds` once. This registers `/start`, `/newpoll`, `/mypolls` and `/help` in Telegram's official command menu (the "☰" button in the chat), so your users get a native command list too. The first `/start` also silently boots the **watchdog** — the hourly background chain that powers auto re-verification and auto-cleaning — so there is nothing else to configure.

✅ **Launch checklist:** bubble renders with 4 buttons · Help page opens in the same bubble · `/setcmds` confirms · bot is admin in your target channel · bot is admin in each sponsor channel you plan to use.

CHAPTER 06

Creating Your First Poll

Tap 🆕 **Create Poll** (or send `/newpoll`). The builder walks you through five short steps, and you can tap ✖ **Cancel** at any moment — the command `/npc` also resets the builder from anywhere. If you close Telegram halfway through, the bot remembers your progress and offers ▶️ **Resume Builder** on the next `/start`.

1. **Title**
   Just type it as a normal message — 3 to 100 characters. No prefixes, no symbols.

2. **Description (optional)**
   A one-line subtitle shown under the title. Tap ⏭ **Skip** if you don't need one.

3. **Options — the cart 🛒**
   Send each option as its own message (1–32 characters). Every send instantly appears numbered in the cart, with 🗑 **Remove last** and ✅ **Done** buttons below. Two options minimum, ten maximum — at ten the bot continues automatically.

4. **Posting channel 📣**
   Send `@yourchannel` or a private channel ID (`-100…`). The bot resolves it live, so typos are caught here rather than at publish time.

5. **Sponsors 🔒 & preview**
   Attach up to two public sponsor channels (`@name`), or skip. The 👀 **Preview** screen shows the exact poll message — tap 🚀 **Publish** when you're happy. The bot resolves every channel, posts to your channel and opens the dashboard.

The published poll message looks like this — buttons are attached directly to the message, and the bars re-render on every vote:



CHAPTER 07

Sponsor Gateway & Join-Left Protection 🔒

When a poll has sponsor channels, voting becomes a two-phase flow. On the first tap the bot pauses and verifies the voter against *every* sponsor channel using Telegram's `getChatMember`. Members pass straight through — their vote is counted and the result bars update everywhere within a second. Non-members get an instant pop-up plus a private DM listing each sponsor channel as a 📣 **Join** button with a big ✅ **I Joined — Check Again** button underneath; tapping it re-runs the same gate and, once they really joined, counts their original choice.

Protection continues after the vote. Three layers keep the tally honest: an **interaction re-check** (any voter tapping 🔄 Refresh who has since left a sponsor channel loses the vote immediately), a **scheduled sweep** (the watchdog re-verifies every live poll's voters hourly by default, five voters at a time to stay light on resources), and a **manual re-verify** (the 🛡 button in the dashboard sweeps on demand). Removed votes decrement the visible totals, increment the 🛡 removed counter and, when a sweep removes anything, the owner receives a private report DM.

> 💡 **Good to know:** gateway DMs can only reach users who have pressed START on your bot at least once — Telegram's rule for all bots. Ask voters to start the bot (the poll's pop-up tells them too); users who never started still see the pop-up but get no DM. The bot never removes a vote when Telegram answers with an API error — only a definite *left* or *kicked* status counts, so temporary outages can't punish honest voters.

CHAPTER 08

Managing Polls — the Dashboard 📊

Everything lives behind 📊 **My Polls** (`/mypolls`), paginated five per page with a live status icon and vote count per poll. Tapping a poll opens its control panel; every action below updates *all posted copies* of the poll message immediately, so your channel never shows stale state.

Action	What happens
⏸ Pause / ▶️ Resume	Voting buttons stop accepting taps with a "paused" pop-up; the banner on the message flips. Resuming re-queues verification soon after.
🙈 Hide / 👁 Show votes	All counts and percentages become `•••` on every copy of the message — one tap, all channels — and back again.
⏹ Stop	Ends the poll permanently: vote buttons disappear, an ENDED banner is applied, sweeps stop, and the data self-deletes 48 hours later.
📈 Stats	Per-option counts and percentages, 🏆 leader marker after the poll ends, unique voters, removed-vote counter, last sweep time.
🛡 Re-verify	Starts an immediate full membership sweep of all voters (chunked in fives); you get a DM report with the removal count.
📣 Re-post	Posts a fresh copy of the poll message to the channel — handy if the original was deleted or pushed down by chat activity.
🗑 Delete	Removes the poll and *all* its data everywhere, instantly and unrecoverably.

CHAPTER 09

Configuration & Data ⚙️

All tunables live in one bot property named `cfg`, created with safe defaults on first start. Edit it in the app under **Bot → Properties** (it is stored as JSON) — or simply leave it alone; every value has a sensible fallback baked in. After any change, send `/start` once so running components pick up the new values.

Property	Default	Meaning
`max_options`	10	Maximum options per poll (2–10 enforced in the builder).
`sweep_interval_min`	60	Minutes between automatic re-verification sweeps of each live poll.
`sweep_chunk`	5	Voters verified per sweep batch before a short yield — keeps iteration usage flat.
`stop_clean_hours`	48	Hours a stopped poll's data is kept before auto-deletion.
`max_age_days`	30	Maximum lifetime of a live poll before the watchdog stops it automatically.

Data is stored in plainly-named properties so you can always audit it in the same Properties screen: `poll_<id>` holds each poll's full record (options, sponsors,

status, votes, posted message ids), `polls_index` lists poll ids for the watchdog, and `u<userid>_np` / `u<userid>_mypolls` keep per-user builder state and dashboard entries. Deleting a property by hand is always safe — the bot treats missing data gracefully.

CHAPTER 10

Troubleshooting 🔧

Almost every issue traced by beta testers came down to one of the rows below. Work top to bottom: check the bot's error log in the app (Bot → Errors) alongside these fixes, since failed API calls are stored with a short diagnostic there. If something truly odd happens, `/start` is always a safe reset — it never destroys poll data.

Symptom	Likely cause	Fix
Bot doesn't reply to `/start`	Token missing/invalid, or import landed on a different bot	Re-paste the BotFather token in the app; confirm the command list actually contains `/start`.
“Couldn't publish the poll”	Bot lacks *Post Messages* admin right, or the channel handle is wrong	Add the bot as admin to the posting channel, check the handle, then tap 🚀 Retry — the draft is kept.
Every vote answers “verification unavailable”	Bot is not an admin in a sponsor channel	Add it as admin there; the gateway refuses to guess when Telegram won't disclose membership.
Voter says no DM arrived	User never pressed START on the bot	Ask them to open the bot once and press START, then tap the check button again.
Poll message doesn't update	The message was deleted, or the bot lost posting rights	Use 🎣 Re-post from the dashboard to publish a fresh copy; verify admin rights if it fails again.
Builder ignores typed text	need_reply flag off (manual paste route)	Open the six builder commands and enable *Wait for answer*; see Route B.
Dashboard list is empty	You're a different BBS user than the owner, or polls were deleted	Dashboards are per-account; publish from the same account, or check the Properties screen for `poll_` records.
Nothing updates for hours	Watchdog chain interrupted (bot paused, plan quota, long downtime)	Send `/start` — it re-boots the watchdog; missed sweeps simply run at the next tick.

CHAPTER 11

Limits & Good Practices

The bot is engineered for the realities of the Bots.Business platform. Votes are stored as a compact map inside each poll's property, which comfortably serves polls up to roughly two thousand voters; beyond that, per-option totals still render but each render pass costs more iterations, so for mega-polls prefer shorter sweep intervals off and rely on the interaction re-check. The chunked sweep design (five voters per batch with short yields) keeps even fully-loaded polls

gentle on your 10-million monthly iteration budget — a quiet day of gateway traffic costs a few thousand iterations at most.

A few practices make the experience better for everyone. Keep option texts short (they become button labels) and titles punchy. Use public channels for sponsors so the gateway can build one-tap join buttons. Tell voters to start the bot before voting day. And when a poll's purpose is served, stop it rather than letting it run to the 30-day cap — stopped polls stay visible in your dashboard for 48 hours with full stats, then vanish from storage automatically, which is exactly what the auto-cleaning database is for.

🧭 **Where to go next:** the platform documentation at `help.bots.business` covers BJS, Git imports, properties and error reading in depth; the `/developers` hub documents API workflows; and the built-in **Settings → Support chat** in the app connects you to the community when you want an extra pair of eyes.

Ready for launch. 🚀

Import the pack, connect your token, add the bot as an admin in your channels — and your first premium poll is one conversation away. Every menu, vote and sweep in this guide was verified by an automated 37-check test suite before packaging.

- **Guides** — bots.business/guides
- **Technical Help** — help.bots.business
- **Developers & API** — bots.business/developers · dev.bots.business
- **In-app support** — Settings → Support chat